

Module 8 — Applied Lab: Vector Retrieval

In this lab you will build the retrieval layer of a RAG system end-to-end on a real-world technical-Q&A corpus. You will define a Weaviate schema with multi-property BM25 indexing, ingest ~1,000–1,500 Stack Exchange posts with externally-supplied embeddings, implement three retrievers (BM25, dense, hybrid), and evaluate all three against a 50-pair labeled set with `recall@5`, `recall@10`, and MRR. The lab closes the gap between “I read about retrieval modes” and “I can choose, implement, and measure a retriever for a real corpus.”

Due: Tuesday end of day. Resubmissions are accepted through Saturday of the assignment week.

What You Will Produce

Committed to your repo:

- retrieval.py** — your implementation of `create_schema`, `index_corpus`, `bm25_search`, `dense_search`, `hybrid_search`, `evaluate_retriever`.
- comparison_brief.md** — a 300–500-word brief reporting your three retrievers’ metrics, identifying queries where each mode wins, and recommending an `alpha` for hybrid.
- setup-notes.md** — short notes on any setup issues you hit (Weaviate container, embedding download, environment) — useful for your own future-you and for the cohort retro.

The starter repo bundles the corpus (`data/corpus.jsonl`, ~1,000–1,500 posts) and the labeled retrieval set (`data/retrieval_eval.jsonl`, 60 query / gold-doc pairs).

Setup

Submission flow — all Module 8 assignments: Module 8 uses a fork-and-submit flow (drill, lab, integration, both stretches) instead of GitHub Classroom. Steps 1–3 below are the new part; steps 4–7 are the same workflow you used in the drill.

- For the template repo to your GitHub account.** Open github.com/LevelUp-Applied-Alim8-8 and click **Fork** (top right) — set the owner to your personal GitHub account → click **Create fork**. Leave “Copy the main branch only” checked.
- Enable Actions on your fork.** On your forked repo, click the **Actions** tab. GitHub disables workflows on forks by default — click **I understand my workflows, go ahead and enable them**. (One-time per fork. The autograder cannot run on your PR until this is done.)
- Clone your fork (not the upstream repo):**

```
git clone https://github.com/<your-github-username>/m8-18.git
cd m8-18
```

- Install dependencies (cached from the drill — should be fast):**

```
pip install -r requirements.txt
```

- Make sure Weaviate is running:**

```
docker ps                                # should list the 'weaviate' c
curl http://localhost:8080/v1/.well-known/ready # should return 200
```

If Weaviate isn’t up, see [Setup: install before the concept session](#) in the reading.

- Create your working branch:**

```
git checkout -b lab-vector-retrieval
```

All implementation work goes in `retrieval.py`. The starter file contains function signatures and docstrings — implement each as described below.

The Corpus

The bundled `data/corpus.jsonl` contains ~1,000–1,500 Stack Exchange technical Q&A posts across the `programmers`, `webmasters`, and `android` subsets, rehydrated from the public Stack Exchange data dump. Each row has the schema:

```
{
  "id": "programmers:12345",
  "subset": "programmers",
  "title": "How do I refactor a long function?",
  "question_text": "I have a function that's grown to 300 lines...",
  "answer_text": "I start by extracting one cohesive responsibility at a time...",
  "text": "How do I refactor a long function?\n\n<question_text>\n\n<answer> <ans
  \"token_count\": 412,
  \"source_url\": \"https://...\",
  \"license\": \"CC BY-SA\",
  \"license_version\": \"CC BY-SA 3.0\"
}
```

`id` is globally unique in `{subset}:{post_id}` form (so all subsets coexist in one Weaviate class). `question_text` and `answer_text` are the structurally-distinct halves of the post; `text` is the concatenation `{\"title\\n\\n<question_text>\\n\\n<answer> {answer_text}\"}` — the dense-embedding source and a backward-compatible “full doc” view.

The starter also bundles `LICENSE` and `ATTRIBUTION.md` per the corpus’s CC-BY-SA share-alike obligation. Do not delete these.

Task 1: Define the Schema

Implement `create_schema(client)`:

The class is `Post`. Properties:

Name	Type	Notes
<code>doc_id</code>	text	filterable; globally-unique <code>{subset}:{post_id}</code>
<code>subset</code>	text	filterable; one of <code>programmers</code> / <code>webmasters</code> / <code>android</code>
<code>title</code>	text	BM25-indexed
<code>question_text</code>	text	BM25-indexed
<code>answer_text</code>	text	BM25-indexed
<code>text</code>	text	stored, not BM25-indexed (used as the dense-embedding source)

Class-level config: `vectorizer: \"none\", vectorIndexConfig: {\"distance\": \"cosine\"}`.

Why three BM25-indexed properties and not just one? BM25 retrieval scores a query against each indexed property. Indexing `title`, `question_text`, and `answer_text` independently gives BM25 the right surface area without double-counting tokens that already appear in the concatenated `text` field. If you BM25-indexed `text` as well, every token would be counted twice (once in the component property, once in `text`), distorting scores.

Mapping the spec to Weaviate v3 property knobs

Weaviate 1.18+ splits the older `indexInverted` flag into two independent property options that together control whether BM25 and filter queries can hit a property.

- indexSearchable: true** — include this property in the BM25 inverted index. Required for `with_bm25(...)` to score against it.
- indexFilterable: true** — include this property in the exact-match filter index. Required for `with_where(...)` to filter on it.
- tokenization: \"word\"** — tokenize on whitespace + punctuation. The cohort default for natural-language text properties; required for sensible BM25 scoring on prose.

For our six properties, the right per-property config is:

Property	indexSearchable	indexFilterable	tokenization	Why
<code>doc_id</code>	false	true	\"field\"	filter target only — exact-match WHERE <code>doc_id = \"...\"</code> , never BM25
<code>subset</code>	false	true	\"field\"	filter target only
<code>title</code>	true	false	\"word\"	one of the three BM25-indexed properties
<code>question_text</code>	true	false	\"word\"	one of the three BM25-indexed properties
<code>answer_text</code>	true	false	\"word\"	one of the three BM25-indexed properties
<code>text</code>	false	false	\"word\"	stored but not indexed — readable from query results, but does not participate in BM25 or filtering. The dense-embedding source.

“stored but not BM25-indexed” operationally means `indexSearchable: false`. The property is retained on every object and returned in query responses; it just doesn’t enter the BM25 inverted index. (`indexFilterable: false` on `text` is also intentional — we never `WHERE`-filter on the full doc string.)

Edge case: if the `Post` class already exists (you’ve run this before), delete it first with `client.schema.delete_class(\"Post\")` before re-creating. Otherwise schema-mismatch errors will surface when you re-ingest with a different shape.

Task 2: Embed and Ingest the Corpus

Implement `index_corpus(client, corpus_path, embedder)`:

- Read every row from `data/corpus.jsonl`.
- Embed the `text` field (the `\"{title}\\n\\n<question_text>\\n\\n<answer> {answer_text}\"` concatenation) with `all-MiniLM-L6-v2`. Batch-encode for speed: `embedder.encode(texts, batch_size=64)`.
- For each row, ingest a Weaviate object with all six properties populated and the externally-supplied vector attached.
- Return the count of ingested objects.

```
def index_corpus(client, corpus_path: str, embedder) -> int:
    """
    Read the corpus, embed each row, and ingest into the Post class.

    Returns:
    int: number of objects ingested.

    """
    # TODO: read JSONL into memory
    # TODO: batch-embed the 'text' field
    # TODO: use client.batch (with 'with_client_batch' or batch.config)
    # TODO: for each row, batch.add_data_object(properties, \"Post\", vector=vec, to
    # TODO: ensure batch is flushed
    # TODO: return count
```

Expect ~1–3 minutes on cohort hardware for the embedding pass. The autograder verifies the final count matches the corpus file’s line count.

Common bugs:

- Forgetting `client.batch.flush()` (or exiting the `with` block) → count = 0.
- Passing the vector as a NumPy array instead of `vec.tolist()` → Weaviate JSON-serialization error.
- Re-using a static class → properties don’t match → ingest silently drops fields.

Task 3: BM25 Retrieval

Implement `bm25_search(client, query, k)`:

```
def bm25_search(client, query: str, k: int) -> list[str]:
    """
    Run a BM25 query against the Post class and return the top-k doc_ids.

    The BM25 search runs over the title, question_text, and answer_text
    properties (the three BM25-indexed properties of the schema).

    Returns:
    list[str]: ordered list of doc_id strings, length ≤ k.

    """
    # TODO: client.query.get(\"Post\", [\"doc_id\"]) with bm25(query=query).with_lim
    # TODO: extract doc_id values from the nested response
```

Return `doc_id` values in the order Weaviate returns them — no re-ordering.

Task 4: Dense Retrieval

Implement `dense_search(client, query, k, embedder)`:

```
def dense_search(client, query: str, k: int, embedder) -> list[str]:
    """
    Embed the query and run a vector-similarity (nearVector) query against
    the Post class. Return the top-k doc_ids.

    Returns:
    list[str]: ordered list of doc_id strings, length ≤ k.

    """
    # TODO: qv = embedder.encode(query).tolist()
    # TODO: client.query.get(\"Post\", [\"doc_id\"]) with near_vector({\"vector\": qv})
    # TODO: extract doc_id values
```

Use the same embedder that you used in `index_corpus`. If you embed the query with a different model than the corpus, the vectors live in a different space and recall collapses to ~0.

Task 5: Hybrid Retrieval

Implement `hybrid_search(client, query, k, embedder, alpha=0.5)`:

```
def hybrid_search(client, query: str, k: int, embedder, alpha: float = 0.5) -> li
    """
    Embed the query and run a hybrid (BM25 + vector) query against the Post
    class with the given alpha. Return the top-k doc_ids.

    Returns:
    list[str]: ordered list of doc_id strings, length ≤ k.

    """
    # TODO: qv = embedder.encode(query).tolist()
    # TODO: client.query.get(\"Post\", [\"doc_id\"]) with hybrid(query=query, vector=
    # TODO: extract doc_id values
```

The autograder fixes `alpha=0.5`. The Tier 1 challenge sweeps `alpha`. Do not use `alpha=0.0` or `alpha=1.0` — the v3 client’s hybrid edge cases at the extremes are unreliable. To get pure-BM25 or pure-vector behavior, call `bm25_search` or `dense_search` directly.

Task 6: Evaluate the Retrievers

Implement `evaluate_retriever(eval_path, search_fn, k_values=(5, 10))`:

For each labeled `{query, gold_doc_id}` pair in `data/retrieval_eval.jsonl`, retrieve the top-`max(k_values)` documents using `search_fn`. Compute:

- recall@5** — fraction of queries where `gold_doc_id` is in the top-5 returned IDs.
- recall@10** — same, top-10.
- mrr** — for each query, contribute `1 / rank_of_gold_doc` (1-indexed) if found in top-10, else 0. Average across queries.

Return a dict like:

```
{
  \"recall@5\": 0.78,
  \"recall@10\": 0.91,
  \"mrr\": 0.62,
  \"by_type\": {
    \"factoid\": {\"recall@5\": ..., \"recall@10\": ..., \"mrr\": ...},
    \"paraphrastic\": {\"recall@5\": ..., \"recall@10\": ..., \"mrr\": ...},
  },
}
```

The `by_type` breakdown — splitting metrics by the labeled set’s `query_type` field (`factoid` vs. `paraphrastic`) — is what makes the comparison brief land. The aggregate number alone hides the BM25-vs-dense divergence; the per-type breakdown reveals it.

Methodology specifications

- recall@k** — A query is “hit at K” if `gold_doc_id` ∈ `top_k_returned_ids`. Returned IDs are taken in the order Weaviate returns them; no re-ordering. Ties in score are broken by Weaviate’s internal ordering.
- MRR** — For each query, find the 1-indexed position of `gold_doc_id` in the returned list of length `max(k_values)` (10 in this lab); contribute `1/rank` if found, 0 if not. MRR is the mean over all queries.
- alpha** — fixed at 0.5 for the hybrid retriever during evaluation. The autograder asserts `alpha=0.5`.
- Top-k for retrieval calls** — retrieve `k=10` for each query; `recall@5` is computed from the top-5 slice of those 10. One retrieval call per query, not two.

Task 7: The Comparison Brief

Write `comparison_brief.md` (~300–500 words). The brief is a short engineering memo to “yourself in three months” arguing for a retrieval choice. Use this structure.

Metrics Table

A table with one row per retriever (BM25, Dense, Hybrid) and the following columns — all required: `recall@5`, `recall@10`, MRR, `factoid recall@5`, `paraphrastic recall@5`. The per-query-type split is what reveals the BM25-vs-dense divergence (see the `by_type` framing earlier in this guide) — the aggregate columns alone hide it.

Where BM25 Wins

Identify at least 2 specific labeled queries where BM25 retrieved the gold document and dense did not (or ranked it much lower). For each, paste the query and explain in one sentence why — what feature of the query made BM25 succeed (rare entity, exact identifier, distinctive token, etc.).

Where Dense Wins

Same structure: at least 2 queries where dense outperformed BM25. Explain in one sentence why for each (paraphrase, vocabulary mismatch, semantic match across surface differences).

Alpha Recommendation

Recommend a hybrid `alpha` for this corpus (a value in `[0, 1]`) and justify in 2–3 sentences. Reasonable values land in `[0.4, 0.6]` for this corpus given the balanced `factoid/paraphrastic` split — but defend your specific number.

Schema choice — cosine vs. dot product

State the schema’s `vectorIndexConfig.distance` value. Confirm whether `all-MiniLM-L6-v2` outputs normalized vectors and what that implies about whether dot product would produce the same ranking on this corpus. Defend the schema’s choice in 1–2 sentences (when does cosine generalize more safely than dot product, even on normalized embeddings?).

Submitting

When all functions are implemented, autograder is green, and the brief is written:

- Run the autograder locally:**

```
pytest tests/ -v
```

All tests should pass. The behavioral floors (`recall@5 ≥ 0.6` for BM25/dense, `≥ 0.7` for hybrid on the fixture subset) are calibrated against a reference implementation; if they fail, the autograder reports which retriever and which fixture queries produced the failure.

- Commit and push:**

```
git add retrieval.py comparison_brief.md setup-notes.md
git commit -m \"Lab 8: BM25 / dense / hybrid retrieval + comparison brief\"
git push origin lab-vector-retrieval
```

- Open a Pull Request within your fork** — base branch: `main`, compare branch: `lab-vector-retrieval`. GitHub defaults to “base repository: LevelUp-Applied-Alim8-8” — change the **base repository** dropdown to your own fork (`<your-github-username>/m8-18`) before opening the PR. The autograder runs against your branch; check the CI status on the PR page to confirm green.

If you see “Workflows are disabled” on the Actions tab, you missed Setup step 2 — enable them and push an empty commit (`git commit --allow-empty -m \"ci: trigger\"`) to re-run. Your PR description must include:

- The `evaluate_retriever` output for **BM25, dense, and hybrid** (`recall@5`, `recall@10`, MRR for each).
- Paste your PR URL into TalentLMS → Module 8 → Lab 8 to submit this assignment.

Challenge Extensions

Optional. The base lab is complete without them. Each tier is independent — pick whichever interests you.

Tier 1 — Alpha Sweep and Per-Query Mode Selection

- Sweep `alpha` over `[0.0, 0.1, 0.2, ..., 1.0]`. Plot `recall@5` vs. `alpha`.
- For each labeled query, identify which `alpha` produced the best result. Are there queries where one `alpha` dominates? Are there queries where global `alpha` is the right choice for this corpus, or whether a per-query routing layer would do better. (The Tuesday stretch picks this thread up.)

Tier 2 — Reciprocal Rank Fusion (RRF) vs. Weaviate Alpha Hybrid

- Implement Reciprocal Rank Fusion: independently retrieve top-50 with `bm25_search` and top-50 with `dense_search`; fuse the two ranked lists using `RRF_score(doc) = Σ 1/(k_rank@k)`. `rank` in RRF) with `k_constant = 60` (the canonical default); take top-10.
- Compare RRF to Weaviate’s `alpha`-weighted hybrid on the labeled set (`recall@5`, `recall@10`, MRR).
- Identify queries where the two hybridization strategies disagree. Explain at least 2 disagreements concretely (RRF rewards documents that rank well in both lists; `alpha`-blend rewards documents with high score in either list).
- Write a 1-paragraph summary on when to prefer score-blending (`alpha`) vs. rank-blending (RRF).

Tier 3 — Custom BM25 Implementation

- Implement BM25 from scratch in NumPy: term frequencies, IDF, document-length normalization. No external library.
- Compare your BM25 to Weaviate’s BM25 on the labeled set. Identify any divergences and explain them (tokenization differences, parameter defaults, stopword handling, etc.).
- Deliver `custom_bm25.py` and a short comparison report (~250 words).